Protocol Emulator: Architecture 1.0 (final)

Status: **frozen, 27 Oct 2026.** Changes only by explicit decision. Supersedes the 25 Oct draft. It builds on *Sub-core Micro-architecture v0.3* and *Sub-core Technical Reference draft 0.1*.

- Diagram sheet: https://claude.ai/artifact/3yvqtC8UtM8iMniqkXfxay
- Reference model: `pe_arch1_model.zip` (assembler, cycle model, area model, 12 test benches)

Target: Tiny Tapeout, IHP sg13cmos5l, 6×4 tiles (1289.28 × 710.64 µm). Pins: ui_in[7:0], uo_out[7:0], uio[7:0], clk, rst_n. Priorities: area first, then timing, then reach into protocols nobody has listed yet.

Contents

0. Summary
1. Architecture summary and protocol fit
2. Area and timing
3. System architecture
4. Sub-core ISA
5. Main core
6. Programming model
7. Example programs
8. Protocol coverage and slot usage
9. Verification status
10. Open items and risks

0. Summary

	v0.3	1.0 draft (25 Oct)	1.0 final
Sub-core	102.1k µm²	74.2k µm²	**81.6k µm²**
Standard cells, chip (+10 % CTS)	553k µm²	400k µm²	**438k µm²**
Density 6×4 / 8×4	63.5 % / 46.9 %	45.9 % / 33.9 %	**50.3 %** / 37.2 %
Instruction	47 bits	32 bits	**32 bits** (unchanged format)
States per sub-core	16	16 (32 ganged)	**16 / 24 / 32** (elastic pairs)
Pins changed by one instruction	2 (two pin slots)	1 output, or one code to a group	**1 output, or six outputs with six different codes**
Data bits per shift event	—	4	**8** (SA lanes + pad taps on SC0, SC2)
Programs verified in the model	—	5	**12** (101 states)

Density uses the deck's definition: standard-cell area × 1.10 / (die − SRAM macro).

1. Architecture summary and protocol fit

1.1 The architecture in brief

The chip splits every protocol into three layers of work, each done where it is cheapest.

- **Host** (off chip, over a 4-pin SPI link). Loads programs and configuration, exchanges data through SRAM buffers, and can reach every register at any time.
- **Main core** (16-bit RISC, program and data in one 512×16 SRAM). Does everything that is byte- or packet-level and not time-critical to the clock: building frames (start, stop, parity, addresses), CRC (with a bit-serial unit up to 32 bits), bit stuffing, NRZI and other line codes, decoding received words, retries and error recovery. It sleeps on `WAIT` until a FIFO needs feeding or draining.
- **Four sub-cores** (each 16 states of 32-bit instructions in latches). Do everything that must be exact to the clock: drive and sample pins, generate and follow protocol clocks, time pulses and gaps, and decide what

happens next within a few clocks of a pin event.

A sub-core instruction is one state of a binary-branch state machine: `COND ? T : F`, plus one pin action and two micro-ops that fire once when the state is entered. The program decides *what happens next*; hardware decides *when*. Every time interval comes from a counter (two 16-bit timers, a 16-bit up/down counter, shifter bit counts), never from counting instructions, so timing does not depend on how the program is written. Each sub-core has:

- 4 event units that turn any pin into a level, rise, fall or either-edge event, optionally qualified by a second pin and optionally sticky
- 2 timers (period, compare, reload, restart-on-event, one-shot, burst) whose waves can drive pins directly
- a counter that loads from a preset, from FIFO A or from the RX shifter, counts on events, stops a burst timer at zero, and can be captured into FIFO B
- a TX shifter (SA) and an RX shifter (SB), 1–16 bits per word, 1/2/4/8 lanes, either bit order, streaming back to back from and into the FIFOs
- FIFO A (4 words, toward the pins) and FIFO B (2 words, from the pins), each word with a tag bit for in-band commands and status
- a value/mask matcher on SB, an EXT condition that ORs up to 10 sources, a 4- or 8-way DISP branch, and two preemption entries
- 6 logical outputs, each following a fixed value, an SA lane, or an AUX source (timer waves, Manchester, inverted partner, received bit, pin level), plus 5 pin patterns that set all six in one clock

Pairs of sub-cores can lend program rows to each other (24 or 32 states), two sub-cores can export 8 SA lanes straight to the pads, and all four can start on the same clock and signal each other, so larger protocols can use more than one engine.

1.2 What protocols need, and which mechanism provides it

Protocol need	Mechanism	Used by
Exact bit timing and baud generation	T1/T2 period counters, chip-wide prescaler (÷1 to ÷4096), 8-bit fractional period on T1, reload value to cancel pipeline offsets	All
Generating a clock and stopping after exactly N cycles	Timer wave on an output (AUX CLK1/CLK2) in burst mode; the counter counts cycles and stops the timer at zero	SPI controller, JTAG, SWD
Following an external clock	Event units on the clock pin; shifters can shift or sample on E0–E3 instead of a timer	SPI target, I2C, PS/2
Sampling at the bit centre and resynchronising	A timer restarted in hardware by every edge (`rst=E0`) gives a half-bit point; T1 then samples each bit centre	UART RX
Serialising and deserialising	SA/SB: N bits per word, MSB or LSB first, stream mode reloads and auto-pushes with no gap	All serial
Any frame format	The main core builds the whole frame as a word (start, data, parity, stop, address, flags); the sub-core only shifts N bits	UART (any data/parity/stop), I2C words
Several data pins per bit	SA lanes: 2, 4 or 8 independent bits per shift event, one output per lane	JTAG TMS/TDI, USB D+/D−, Quad-SPI, 8080
Bidirectional and open-drain lines	Open-drain pad mode, release (Z) pin code, and drive-enable taken from a second SA lane so direction changes are part of the data stream	I2C, 1-Wire, SWD, MDIO, 3-wire SPI
Several control pins changing on one edge	Pin patterns (six codes in one clock), lane group, all-idle / all-release, same-clock inverted partner	8080 D/C_n + CS_n, USB release, differential strobes
8-bit parallel buses	8 SA lanes exported to 8 pads as registered taps (SC0, SC2)	8080, 6800, LCD
Real-time decisions on received data	32 conditions (events, timers, shifter status, counter zero, current bit, matcher, FIFO status, any pin level); DISP for 2–3 bit fields; matcher for addresses and sync words	I2C arbitration, SWD ACK, address match

Protocol need	Mechanism	Used by
Waiting for whichever comes first	EXT OR-mask over 10 sources; sticky events so a short pulse is never missed	NEC (next mark or timeout)
Aborting from anywhere	Two preemption entries (always, or only in marked states) on an event, an error or EXT	SPI target CS_n rise
Measuring pulse widths and positions	Counter counting a tick or qualified events up/down; CAP pushes the value into FIFO B	NEC IR, SENT RX, quadrature
Lengths decided at run time	Counter loaded from FIFO A (bit or cycle counts per transaction); tagged FIFO A words select the transaction type	JTAG, SWD, I2C word types
Reporting errors with the data	MARK puts a tagged word (the current state) into FIFO B in order with the data; HALT plus soft reset and FIFO flush for recovery	UART framing error, SWD WAIT, I2C arbitration loss
Long state machines	Elastic pairs: 24 or 32 states for the master, the partner keeps 8 or none	I2C multi-controller
Protocols spread over engines	8 SIG lines between sub-cores, same-clock start (RUN_MASK), mailbox bits from the main core	Cooperating engines

1.3 How each protocol class maps

Each class below includes protocols already run in the cycle model (§7) and protocols expected to fit but not yet modelled; §8 marks which is which.

Asynchronous serial (UART, LIN, DMX512, ISO 7816). Transmit needs only two states: the main core builds each frame as a word with start, data, parity and stop bits, and SA streams words back to back, shifting on T1's period. Because the frame is just data, any data width, parity or stop-bit count up to 16 bits per word works without touching the program. Receive waits for the start edge, uses a timer restarted by that edge to check the middle of the start bit (rejecting glitches), then samples every bit centre with T1 and collects bits in SB. A bad stop bit adds a tagged MARK word after the byte, so the main core sees exactly which byte failed. Break conditions and guard times are one more timer wait.

Controller-clocked synchronous buses (SPI controller, 3-wire SPI, Microwire, Quad-SPI, JTAG, SWD, MDIO). The clock pin follows a timer wave in burst mode, and the counter stops it after exactly the number of cycles loaded from the preset or from FIFO A. SA changes data on the wave's falling side (period event) and SB samples on the rising side (compare event), so setup and hold are fixed by timer values. Chip select is a single pin action. Everything else is data: in JTAG, TMS and TDI ride two SA lanes, so walking the TAP state machine is just a bit pattern and the same 5-state program does every scan. On shared data lines (SWD, MDIO, 3-wire SPI), the second lane carries the drive enable, so turnarounds happen exactly where the main core put them in the stream. Where the controller must react to the target (SWD's 3-bit ACK), one DISP state branches four ways within a few clocks, well inside one bit period.

Target-side and externally clocked buses (SPI target, PS/2, I2C target). Event units on the incoming clock drive the shifters directly, so receive and reply bits move with no program involvement. A preemption entry on chip-select release aborts from any state and releases the data line. The matcher compares a received address or sync word as soon as its last bit is in, so the sub-core can decide to ACK or ignore within the next bit.

Open-drain, multi-drop buses with handshaking (I2C controller and multi-controller, SMBus, 1-Wire). The program drives only the clock line. The data line always follows SA, and SA shifts on the *real* clock edge seen at the pin, so data never changes in the same clock as the clock line, and START/STOP are a single step while the clock is held high. Clock stretching is handled by waiting for the real rise before timing the high phase (a timer restarted by the rise event). Bus speed is set by timer values only. For multi-controller I2C, the program compares the bit it sent with the pin after each rise; on a mismatch it releases both lines with one pin action, marks the loss and waits for the winner's STOP (an event qualified by the clock being high), while the main core flushes and re-queues. That program needs 24 states, so it borrows 8 rows from its pair partner, which keeps running its own 8-state program. 1-Wire uses the same pieces: open-drain output, timer slots, and SB sampling at the read point.

Line-coded and symbol-level transmit (USB low-speed TX, Manchester, 10BASE-T TX, WS2812, DShot). The main core does the line coding (NRZI, bit stuffing, end-of-packet). Multi-wire symbols such as USB's J, K and SE0 ride two SA lanes, so both lines change on the same edge. The underrun-hold mode keeps the last symbol when SA runs dry, so no illegal state appears between words. Manchester is an AUX source (SA ⊕ timer wave). Pulse-width-coded LEDs branch on the current bit to choose a short or long high time from the two timer events.

Parallel buses (8080, 6800, LCD controllers). SC0 and SC2 export 8 SA lanes to 8 pads, so a whole byte appears on the bus in one clock; their drive enable follows O5 for bus direction. Control lines are the six logical outputs: patterns switch D/C_n and CS_n together, and the write strobe is a pin action timed by T1. Tagged FIFO A words mark command bytes.

Timing-encoded inputs (NEC IR, SENT, echo timing, quadrature encoders). The counter counts a fixed tick, and CAP pushes its value into FIFO B at each edge, so the main core receives a list of widths and classifies them in software. EXT waits for the next edge or a frame timeout in a single state. For a quadrature encoder, two qualified events count up and down in hardware with no program steps; the main core asks for a snapshot through a mailbox bit.

Waveform generation (PWM, servo, stepper STEP/DIR). Timer waves on outputs give PWM with no program activity; patterns set direction and enable lines together; the counter counts steps.

1.4 Why one pin action and two micro-ops per state are enough

- **Most per-bit work is not done by instructions.** Shifting, sampling, clock generation, cycle counting, pulse capture and pin-following all run in hardware once configured. Instructions only mark the points where the protocol makes a decision or changes phase.
- **Measured over 12 protocols (101 states):** 33 states fire nothing (pure waits and branches), 44 fire only micro-ops, and 24 fire one pin action. No state needed a second pin action or a third micro-op.
- **When several pins must change together**, it is almost always either data (SA lanes, which carry any values) or a fixed control combination (a pattern). Outputs following SA or AUX change on the same edge as a pin action, so one state can move a strobe while data and clock pins change too.
- **Program size:** typical programs need 2–16 states. The largest, multi-controller I2C with arbitration, needs 24 and fits by borrowing rows from its pair partner.
- **Timing never depends on the program.** One state per clock with no stalls, and fixed latencies (action to pin 1 clock, event to pin 3 clocks, pad to pad 5 clocks) that the assembler folds into timer constants. A decision after an event takes a few clocks, much less than one bit at the rates targeted.

1.5 Speed envelope and limits

- **Controller-clocked:** the model runs SPI at clk/4 (12.5 MHz at 50 MHz, with 20 ns MOSI setup) and JTAG/SWD at clk/10. Anything faster than clk/10 depends on the CMOS5L pads, which are uncharacterised; the logic is not the limit.
- **Externally clocked:** the input path has a 3-clock latency, so a mode-0 SPI target must get its reply bit to the pin within half an SCK period. That puts the practical limit near clk/8; it is verified at clk/12 (about 4 MHz).
- **Program in the loop every bit** (I2C, 1-Wire): a few clocks of decision time per bit, comfortable up to the low-MHz range; I2C is verified at 100 kHz and about 390 kHz.
- **Sustained data rate** is bounded by the main core feeding FIFO A, about one 16-bit word per 7 clocks in a tight loop; faster streams run in bursts of 4 words.
- **Out of reach or marginal:** USB low-speed *device* (handshake turnaround with a software CRC is tight), CAN with a real-time CRC-dependent ACK, 10BASE-T receive (5 clocks per bit) and quadrature ×2/×4 decoding (one increment and one decrement event only). §8 lists what is verified and what is expected to fit.

2. Area and timing

Sub-core budget (k µm², sg13cmos5l typical liberty; scaled block by block from the v0.3 synthesis, the same method reproduces v0.3's 63.5 %)

Block	Area	Notes
Program store	22.2	16 × 32 dlhrq latches, one lgcp per 16-bit row, 16:1 read mux, half of the pair bank select
FIFOs	9.7	FIFO A 4 × 17, FIFO B 2 × 17, in flops
Timers	6.4	T1 and T2, 16-bit; 8-bit fraction on T1
Shifters	7.3	SA and SB, 16-bit, 1/2/4/8 lanes, underrun hold
Outputs	6.7	6 outputs, lane select, OE from lane, pattern decode
Event units	3.8	4 units with sticky bits
Counter	3.6	16-bit up/down, wrap, CAP path
Sequencer	4.3	32:1 condition mux, 2 micro-op decoders, preemption, DISP
Config decode	3.9	66 word strobes, 8 debug words
Matcher, EXT	1.1	
Config latches	12.6	447 bits, dlhrq, plus the rst_n buffer tree
Total	**81.6**	

Chip budget

Block	Area
4 sub-cores	326.2k
Pin layer (43:1 pad source select × 15, 16 SA taps, sync, filters)	26.3k
Main core (with the 32-bit CRC unit)	32.0k
Host link, system controller, bus	14.0k
Subtotal × 1.10 (CTS)	**438.4k**
SRAM macro RM_IHPSG13_1P_512×16 (excluded from density)	45.3k
Density 6×4 / 8×4	**50.3 % / 37.2 %**

Sensitivity of the 6×4 density to the estimate: ×0.85 → 42.8 %, ×1.15 → 57.9 %, ×1.30 → 65.4 %. If hardening comes in high, the levers in §10 apply in order.

Design options weighed

Option	Density 6×4	Why not
24 states in every sub-core	55.8 %	+5.5 points for capacity most programs never use. The elastic store gives 24 or 32 states where needed for ≈0.5k
32 states in every sub-core	61.3 %	same, +11 points
FIFO 4/4	51.4 %	RX words arrive at most one per N bit times; FIFO B depth 2 was enough in every test
A second pin slot (38-bit instruction)	52.4 % (50.7 % if it replaced patterns)	+4.2k per sub-core (6 latch bits per row plus a wider read mux) and still only two pins per state. A pattern changes all six outputs for 3.5k, and no state in the 12 programs needs two independent pin actions that a pattern or SA lanes do not cover
Wider OSEL/POP, or a third micro-op slot	—	Across the 101 states of the 12 final programs, no state needed more than 1 pin action + 2 micro-ops (table in §8)

Timing - One clock domain. 50 MHz target; 48 MHz gives USB-LS exactly 32 clocks per bit; 40 MHz suits 10BASE-T TX. - **Sub-core next-state loop:** state register → 16:1 latch read (two 8:1 halves + 2:1) → own/partner 2:1 (pair masters only) → 32:1 condition mux → T/F 2:1 → DISP OR → two preemption 2:1 → state register. v0.3 measured 1.2 ns typ / 2.0 ns slow for a wider version of this loop without the pair select and DISP. The estimate with them is under 3 ns slow, against a 20 ns period. DISP and pattern sources are all registered or static config, so they add no new start points. - **Pair placement:** the partner bank's read data crosses into the master's next-state loop, so SC0/SC1 and SC2/SC3 are placed side by side (floorplan constraint). - **Main core:** the longest path is likely SRAM clock-to-out (3.7–3.9 ns typ, 6.3–6.6 ns slow), then decode, register read and the 16-bit ALU. Estimated 12–14 ns slow. - **Latency contract:** a state's action reaches a pin in 1 clock; any event reaches a pin in 3 clocks; pad to pad is 5 clocks (100 ns at 50 MHz). Pattern, lane, tap and NPART outputs all obey the same 1-clock action rule, so they land on the same edge.

---

3. System architecture

- **Host link.** SPI target, mode 0, on ui0–2 (SCK, CS_n, MOSI) and uo0 (MISO). It is bus master 0 and always wins arbitration. Commands are op + 16-bit address + data words with auto-increment. The host can reach every register at any time.
- **System bus.** 16-bit, word-addressed. Masters in priority order: host link, then main core. Writes take 1 clock; reads take 2 (registered read data).
- **Main core.** 16-bit RISC, r0 = 0 and r1–r7, 16-bit instructions, one per clock. Fetch and data share the SRAM port: an SRAM load or store adds one clock, and so does a taken branch. `WAIT` sleeps until a masked bit of the event vector is set. Bit-serial CRC unit up to 32 bits at 0x1C10. Allocation 32k µm².
- **SRAM.** RM_IHPSG13_1P_512×16, 45.3k µm², the only macro on the chip.
- **System controller.** Reset sequence (everything halted, every pad 'none'); RUN_MASK, which starts several sub-cores on the same clock; host IRQ and doorbell; the shared prescaler taps (÷1, ÷16, ÷256, ÷4096); the ID register; and collection of sub-core status into the main core's WAIT vector.
- **Sub-cores.** Four, 81.6k µm² each. Each has:
- a 16 × 32-bit latch program bank and a sequencer (one state per clock, no stalls)
- 4 event units (pin, level/rise/fall/both, qualifier pin, pulse or sticky)
- T1 and T2: 16-bit timers with PER, CMP, RLD, restart-on-event, one-shot and burst modes, and a wave output; T1 adds an 8-bit fraction
- a 16-bit up/down counter that can stop a burst timer and be captured into FIFO B
- SA (TX) and SB (RX) 16-bit shifters: N 1–16, MSB- or LSB-first, 1/2/4/8 lanes, stream mode
- FIFO A (4 × 17 bits, toward the pins) and FIFO B (2 × 17 bits, from the pins); bit 16 is a tag
- a 16-bit value/mask matcher on SB
- 6 logical outputs and 5 pin patterns
- **Elastic pairs.** SC0+SC1 and SC2+SC3 share their 32 latch rows as 16/16, 24/8 or 32/0 (§4.10).
- **Inter-engine signals.** Each sub-core drives two SIG pulse lines (8 chip-wide); any sub-core can wait on one through EXT. With RUN_MASK, engines start on the same clock and stay cycle-locked.
- **Pin layer, outputs.** 15 output-capable pads: uo1–uo7 and uio0–uio7 (uo0 is host MISO). Each pad has a 43:1 source select (none, GPIO, IRQ, the 24 logical outputs, the 16 SA taps), a push-pull / open-drain / input mode, and resettable config. uo pads have no output enable in the TT harness, so a released output on a uo pad keeps driving its value; open-drain buses (I2C, 1-Wire, SWDIO) go on uio pads.
- **Pin layer, inputs.** 16 input pads: ui0–7 and uio0–7. Each passes a 2-FF synchroniser, a 0–3 clock glitch filter and a loopback mux, then one shared IN_R register.

Pins per clock - **One instruction** changes one output (OSEL 0–5); or puts one code on the lane group (OSEL 6); or applies a pattern (OSEL 7, POP 1–5), which gives each of the six outputs its own code (drive 0, drive 1, release, toggle, follow SA, follow AUX, back to idle) in one clock; or releases all six, or returns all six to idle. - **Every clock, with no instruction:** outputs that follow SA or AUX change with their source. With 8 lanes, SA presents 8 independent bits per shift event. - **SA taps:** on SC0 and SC2, SA lanes 0–7 also reach the pad select

directly as 8 registered taps whose drive enable is O5's. One sub-core can drive an 8-bit bus plus six control outputs, 14 pads on one edge (checked in the model). - **Whole chip:** 24 logical outputs and 16 taps feed 15 pads, and one source can fan out to several pads. The main core can drive any GPIO-owned pad with one store, but that timing follows the software.

---

4. Sub-core ISA

4.1 Instruction, 32 bits

Bits	Field	Meaning
31:27	COND	Condition to test, C0–C31
26:22	T	Next state if true (0–31)
21:17	F	Next state if false (T = F is a plain jump)
16	PE	Enables the gated preemption entries while in this state
15:13	OSEL	0–5: one output; 6: the lane group; 7: group operation (pattern, all release, all idle)
12:10	POP	With OSEL 0–6: pin code (§4.4). With OSEL 7: 1–5 = PAT1–PAT5, 6 = all release, 7 = all idle
9:5	OPA	Micro-op
4:0	OPB	Micro-op

4.2 Execution rules

- One state per clock, never a stall: `next = P0 hit ? vec0 : P1 hit ? vec1 : (C[COND] ? T : F) | (DISP ? field : 0)`
- A preemption entry fires on its event when it is marked *always*, or when it is *gated* and the current state has PE set. P0 has priority over P1.
- **Actions fire once**, in the first clock of a state entered from a different state, or entered by preemption or reset. Waiting with F = self does not fire them again.
- A condition sees each resource as it was *before* the current state's entry actions take effect. A test that depends on an entry action belongs in the next state.
- Output sources, shifter arming and timer run state persist after the state is left.
- A sticky condition (sticky event unit, sticky EXT) clears when a branch that tested it true is taken, or on its CLR micro-op.
- When two micro-ops in one state touch the same resource, the first in this order wins: SA: LOAD, STOP, STEP/shift; SB: ARM, PUSH, STOP, STEP/capture; CNT: LDP, LDF, LDB, then INC/DEC (both together cancel).
- **Halted** (HALT micro-op, CTRL.HALT, or reaching the breakpoint state): the state holds and no actions fire. Timers and armed shifters keep their mode, so stop them before HALT if it matters. CTRL.STEP runs one sequencer clock. Program and config may be written only while halted.

4.3 Conditions (5 bits, all 32 used)

C	Name	Source
0–3	E0–E3	Event units (pulse, or sticky level)
4–7	T1P T1C T2P T2C	Timer period / compare events
8	SAD	SA empty (finished, or nothing loaded)
9	SBD	SB holds N bits (non-stream mode)
10	CNT0	Counter = 0
11	SBIT	SA's current bit (lane 0), or SB's last sampled bit (config)
12	MATCH	((SB ⊕ value) & mask) == 0

C	Name	Source
13	TXNE	FIFO A not empty
14	TXTAG	FIFO A head is tagged
15	EXT	OR-mask of selected sources (§4.8), level or sticky
16–31	pin	IN_R[0..15] = ui0–7, uio0–7

4.4 Pin slot

Pin codes (POP with OSEL 0–6, and each 3-bit entry of a pattern)

Code	Assembler	Effect on the output
0	—	No change
1	`Ok=0`	Drive 0
2	`Ok=1`	Drive 1
3	`Ok=Z`	Release (drive enable 0)
4	`Ok^`	Toggle (and drive)
5	`Ok=SA`	Follow SA (its lane), driven
6	`Ok=AUX`	Follow its AUX source, driven
7	`Ok=IDLE`	Return to its initial source and idle value

OSEL - 0–5: the code applies to output O*k*. - 6 (`LANES=code`): the code applies to O0..O(min(lanes, 6) − 1). - 7: POP 1–5 applies pattern PAT1–PAT5 (`PATk`); POP 6 releases all six (`ALL=Z`); POP 7 returns all six to idle (`ALL=IDLE`).

A **pattern** is 18 config bits: one pin code for each of O0–O5. All six outputs take their codes on the same clock edge. Code 0 leaves an output alone, so a pattern can touch any subset.

4.5 Micro-ops (5 bits, all 32 used)

#	Name	Effect
0	NOP	—
1	SA.LOAD	Arm SA and load it from FIFO A (if FIFO A is empty, SA stays armed and empty; stream mode loads on the next shift event)
2	SA.STEP	One shift now, as if a shift event occurred
3	SA.STOP	Disarm SA; SA becomes empty
4	SB.ARM	Clear SB and arm it
5	SB.STEP	One sample now
6	SB.PUSH	Push SB's partial word (right-aligned) into FIFO B and clear SB; no-op if SB is empty
7	SB.STOP	Disarm SB
8	T1.GO	T1 count = RLD, run (restart)
9	T1.STOP	Stop T1
10	T2.GO	T2 count = RLD, run
11	T2.STOP	Stop T2
12	CNT.LDP	CNT = preset
13	CNT.LDF	CNT = FIFO A head (popped); error FA_EMPTY if empty
14	CNT.LDB	CNT = SB
15	CNT.DEC	CNT − 1
16–19	CLR.E0–E3	Clear event unit k's sticky bit

#	Name	Effect
20	CLR.EXT	Clear the sticky EXT bit
21	SIG0	One-clock pulse on this sub-core's SIG line 2n
22	SIG1	One-clock pulse on SIG line 2n+1
23–25	NTF1–NTF3	Set notify flag 1–3 (main-core WAIT bit, host-visible)
26	MARK	Push a tagged word (the current state number) into FIFO B
27	SA.SKIP	Drop the FIFO A head
28	HALT	Halt this sub-core
29	CNT.INC	CNT + 1
30	CAP	Push CNT into FIFO B (input capture)
31	DISP	Multi-way branch: next = T | field (§4.9)

4.6 Datapath blocks

Event units E0–E3. Pin (any of the 16 inputs); mode level, rise, fall or both edges; optional qualifier (another pin must be at a given level); pulse or sticky. Edges compare IN with IN_R, so an event is aligned with IN_R.

Timers T1, T2 (16-bit) - Count from RLD to PER on each prescaler tick (tap ÷1, ÷16, ÷256 or ÷4096), so a free-running period is PER + 1 ticks. Period event P at the end of a period, compare event C at count = CMP. - The wave output goes high after C and low at P (CLK1, CLK2 in the AUX list). - GO or a restart event: count = RLD, run, wave low. RLD (4 bits) cancels pipeline offsets, so the first period after a GO can be exactly as long as the rest. - Restart event select: E0–E3, the other timer's P or C. - One-shot: stops at P. Burst (gate): stops when the counter reaches 0, and the AUX clock is held low while CNT = 0. - T1 only: an 8-bit fraction. Each period adds FRAC to an accumulator; a carry stretches the next period by one tick, so the average period is PER + 1 + FRAC/256.

Counter (16-bit). Load preset, from FIFO A or from SB; increment and decrement by micro-op or by a selected event (E0–E3, T1P, T2P, SA shift, SB capture); wrap or saturate; CNT0 condition; CAP pushes it into FIFO B; reaching 0 stops a burst timer.

SA (TX shifter, 16-bit) - N = 1–16 bits per word (N a multiple of the lane count); lanes 1, 2, 4 or 8; MSB- or LSB-first. Words are right-aligned: an MSB-first word presents bit N−1 first. With MSB-first, lane 0 is the most significant bit of each group. - Shift event: T1P, T1C, T2P, T2C or E0–E3. Skip-first ignores the first shift event after LOAD. - Stream mode reloads from FIFO A at each word boundary until SA.STOP. Otherwise SA disarms after N bits. - Underrun value (what SA presents while empty): 0, 1, or hold the last lane group. - SAD = empty. SC0 and SC2 also export lanes 0–7 as pad taps (§3).

SB (RX shifter, 16-bit) - N, lanes, bit order and capture event as for SA, sampling IN_R[pin .. pin + lanes − 1]. - Stream mode pushes every N bits into FIFO B automatically. Otherwise SBD is set after N bits and SB disarms. - Complete and partial words are right-aligned.

FIFOs. FIFO A: 4 × 17 bits, written over the bus (data or tagged), read by SA.LOAD / stream reload, CNT.LDF and SA.SKIP; seen by TXNE, TXTAG and DISP. FIFO B: 2 × 17 bits, written by SB, MARK and CAP, read over the bus. FBNEW is a register holding the newest word pushed into FIFO B.

Errors. FA_EMPTY (CNT.LDF on an empty FIFO A), FB_OVR (push to a full FIFO B, word dropped), FA_OVR (bus push to a full FIFO A, word dropped). Flags show in STATUS and clear with CTRL.CLR_ERR. ERR as a preemption event is the clock a new error appears; ERR as an EXT source is "any flag set".

Matcher. ((SB ⊕ value) & mask) == 0, value and mask 16 bits each.

4.7 Outputs O0–O5

Each output has config: idle value, invert, AUX source, initial source, lane, OE-from-lane.

Field	Choices
Initial source (soft reset, and pin code 7)	IDLE (drive the idle value), SA, AUX, Z (released)
AUX source	CLK1 (T1 wave), CLK2 (T2 wave), MANCH (SA ⊕ CLK1), NPART (¬ the partner output O(k xor 1), same clock), SBBIT (SB's last sample), E0LVL, E1LVL (the event units' pin levels), ZERO
Lane	Which SA lane it follows (default k mod lanes)
OE from lane	Drive enable taken from SA lane + 1, so a lane pair carries (value, drive enable) per bit time

Value and drive enable are registered in the output stage. That is the 1 clock of the latency contract.

4.8 EXT condition

EXT = OR of the enabled sources {E0, E1, E2, E3, T1P, T2P, MB[sel], SIG[sel], RXFULL, ERR}, where MB[sel] is one of the 4 mailbox bits (written by the main core or host) and SIG[sel] is one of the 8 SIG lines. In sticky mode EXT latches until a taken branch on EXT or CLR.EXT. EXT is also a preemption event.

4.9 DISP (multi-way branch)

- Config: source (SB, FBNEW, FAHEAD = FIFO A head data, IN = the 16 pins), base bit (0–15), width 2 or 3.
- In a state that fires DISP: next = T | field. T must be aligned to 4 (width 2) or 8 (width 3), and the state must be unconditional (T = F). The assembler checks both. Preemption still wins.
- Example: SWD reads 12 bits into SB, then one DISP state sends OK, WAIT, FAULT and no-target to four different states. From SB done to the right state takes about 3 clocks, well inside one SWCLK period (10 clocks).

4.10 Elastic program store

- SC0+SC1 and SC2+SC3 each own 32 latch rows (16 per bank). The pair master (SC0, SC2) selects the split in its config:

Split	Master states	Partner states	Partner
16/16	0–15 (own bank)	0–15 (own bank)	independent
24/8	0–15 own + 16–23 = partner rows 0–7	8–15 (own rows 8–15; assembler `.base 8`)	independent, 8 states
32/0	0–31	none	halted; its datapath stays idle

- **How it is built.** Each bank's 16:1 read mux is two 8:1 halves and a 2:1. In 24/8 the partner's lower half takes its select from the master's state and the partner's final 2:1 is forced to the upper half; in 32/0 the whole partner mux follows the master. The master adds a 32-bit 2:1 (own or partner bank, by state bit 4). The partner keeps its own sequencer and datapath; only the master's 5-bit state can reach rows 16–31.
- **Loading.** Program words are written physically: master state 16 + k is partner row k (partner window 0x40 + 2k). The assembler emits the split image. Both sub-cores must be halted while writing.
- The partner's own program in a 24/8 split must use only states 8–15.

4.11 Configuration map (447 bits, 34 words at window offsets 0x10–0x31)

Written only while halted; all bits clear at rst_n.

Offset	Register	Fields
0x10	T1_PER	[15:0]
0x11	T1_CMP	[15:0]

Offset	Register	Fields
0x12	T1_CTL	[3:0] RLD, [5:4] PSC (÷1/16/256/4096), [6] RST_EN, [9:7] RST_SEL (E0–E3, other P, other C), [10] ONESHOT, [11] GATE
0x13	T1_FRAC	[7:0]
0x14	T2_PER	[15:0]
0x15	T2_CMP	[15:0]
0x16	T2_CTL	as T1_CTL
0x17	CNT_PRE	[15:0] preset
0x18	CNT_CTL	[0] DEC_EN, [3:1] DEC_SEL, [4] INC_EN, [7:5] INC_SEL (E0–E3, T1P, T2P, SA shift, SB capture), [8] WRAP
0x19	MATCH_VAL	[15:0]
0x1A	MATCH_MASK	[15:0]
0x1B	SA_CFG	[3:0] N−1, [4] MSB, [6:5] LANES (1/2/4/8), [9:7] EV (T1P T1C T2P T2C E0–E3), [10] STREAM, [11] SKIP1, [13:12] UDF (0, 1, hold)
0x1C	SB_CFG	[3:0] N−1, [4] MSB, [6:5] LANES, [9:7] EV, [10] STREAM, [14:11] PIN
0x1D–0x20	EVU0–EVU3	[3:0] PIN, [5:4] MODE (level/rise/fall/both), [6] STICKY, [7] QEN, [11:8] QPIN, [12] QLVL
0x21–0x26	OUT0–OUT5	[0] IDLE, [1] INV, [4:2] AUX, [6:5] INIT (IDLE/SA/AUX/Z), [9:7] LANE, [10] OE_LANE
0x27	EXT_CFG	[9:0] MASK (E0 E1 E2 E3 T1P T2P MB SIG RXFULL ERR), [11:10] MB_SEL, [14:12] SIG_SEL, [15] STICKY
0x28	SEQ_CFG	[4:0] ENTRY, [6:5] SPLIT (16/24/32; pair masters only), [7] BP_EN, [12:8] BP_STATE, [13] SBIT_SRC (SA/SB)
0x29	DISP_CFG	[1:0] SRC (SB/FBNEW/FAHEAD/IN), [5:2] BASE, [6] WIDTH3
0x2A, 0x2B	PRE0, PRE1	[0] EN, [3:1] EV (E0–E3, T1P, T2P, ERR, EXT), [8:4] VECTOR, [9] ALWAYS
0x2C–0x31	PAT0–PAT5	Output k's code in each pattern: [2:0] PAT1, [5:3] PAT2, [8:6] PAT3, [11:9] PAT4, [14:12] PAT5

The layout is field-aligned for software; packed, the 447 bits would fit 28 words. The area model counts the extra 6 word strobes.

---

5. Main core

5.1 ISA, 16-bit

Fields: [15:12] op, [11:9] rd, [8:6] rs, [5:3] rt, [2:0] fn; imm6 = [5:0].

Op	Operands	Instructions
0	rd, rs, rt	ADD SUB AND OR XOR SLL SRL SRA (fn)
1–3	rd, rs, imm6	ADDI (signed) · SHI {SLL, SRL, SRA} n ([5:4] type, [3:0] amount) · ANDI
4–5	rd, [rs + uimm6]	LD · ST
6, 7, D	rd, rs, off6	BEQ · BNE · BLTU (unsigned)
8, 9	rd, imm	LI (simm9) · LIH (rd = imm8 << 8 | rd[7:0])
A, B	off12	JMP · JAL (link in r7)
C	system ([5:0] fn)	JR rs · WAIT rd, rs · HALT · IRQ · NOP
E, F		Reserved

Timing: one instruction per clock; +1 clock for an SRAM load/store (shared port), a bus read (registered data) or a taken branch/jump. `WAIT rd, rs` sleeps until (events & rs) ≠ 0 and returns the hit bits in rd.

WAIT event vector

Bits	Meaning
[3:0]	FIFO A has space (one bit per sub-core)
[7:4]	FIFO B has data
[11:8]	Notify pending
[12]	Host doorbell
[13]	CRC unit idle
[14]	A sub-core halted or raised an error

5.2 CRC unit (0x1C10)

Bit-serial LFSR, polynomial up to 32 bits, MSB-first or reflected, 1 bit per clock, 8 bits per DATA write. A DATA write or VAL read issued while busy waits for the unit.

Address	Register
0x1C10 / 0x1C11	POLY_L / POLY_H (reflected polynomial in reflect mode, e.g. 0xEDB88320)
0x1C12	CFG: [4:0] width − 1, [5] reflect
0x1C13	DATA: write a byte, 8 clocks
0x1C14 / 0x1C15	VAL_L / VAL_H: write the initial value, read the result; final XOR is software

Checked: CRC-8/SMBUS 0xF4, CRC-16/ARC 0xBB3D, CRC-32 0xCBF43926.

6. Programming model

Division of work - **Host:** loads everything and exchanges data through SRAM buffers. For simple protocols it can push and pop the FIFOs directly and never start the main core. - **Main core:** turns bytes into bit streams and back (framing, parity, CRC, stuffing, line coding, packet logic), moves words between SRAM and the FIFOs using WAIT, and handles HALT/MARK recovery (soft reset, flush, retry). - **Sub-cores:** drive and sample pins with clock-exact timing.

6.1 Memory map (16-bit word addresses)

Range	Block
0x0000–0x01FF	SRAM: main-core code, data, host buffers
0x1000 + 0x80·n	Sub-core n window, n = 0–3 (0x1000–0x11FF)
0x1800–0x181F	Pin layer
0x1C00–0x1C1F	System block and CRC unit

6.2 Sub-core window

Offset	Register	Access
0x00	CTRL: [0] RUN, [1] HALT, [2] STEP, [3] SOFT_RESET, [4] FLUSH_A, [5] FLUSH_B, [6] CLR_ERR	W
0x01	STATUS: [4:0] state, [5] halted, [6] breakpoint hit, [9:7] NTF1–3, [12:10] FA_EMPTY, FB_OVR, FA_OVR	R
0x02	NTF ack: [2:0] clear NTF1–3	W
0x03	MAILBOX: [3:0] MB0–3	RW
0x04	FIFO A push, data word	W

Offset	Register	Access
0x05	FIFO A push, tagged word	W
0x06	FIFO B pop	R
0x07	LEVELS: [3:0] FIFO A count, [7:4] FIFO B count, [8] FIFO B head tag	R
0x08–0x0F	Debug: flags, CNT, T1 and T2 counts, SA, SB, FIFO A head, event register	R
0x10–0x31	Config (§4.11)	W, halted only
0x40–0x5F	Program: row s at 0x40 + 2s (bits 15:0) and 0x41 + 2s (bits 31:16)	W, halted only

SOFT_RESET: state = entry state, outputs take their initial source, timers stop, shifters disarm, counter and sticky bits clear, both FIFOs flush.

6.3 Pin layer (0x1800)

Offset	Register
0x00–0x0E	PAD_CFG for uo1–uo7, uio0–uio7: [5:0] SRC, [7:6] MODE (push-pull, open-drain, input)
0x10	GPIO_OUT [14:0] (same pad order)
0x11	GPIO_OE [14:0]
0x12	IN [15:0]: synchronised inputs ui0–7, uio0–7 (R)
0x13 / 0x14	FILT_UI / FILT_UIO: 2 bits per input, 0–3 clocks
0x15	LOOPBACK [15:0]: input k reads the output side of its pad (ui k ← uo k, uio k ← its own driver)

Pad source codes (6 bits, 43 used)

Code	Source
0	none (uo drives 0, uio released)
1	GPIO
2	Host IRQ
3–7	reserved
8 + 6n + k	Sub-core n output O*k* (n 0–3, k 0–5) → 8–31
32–39	SC0 SA tap 0–7 (drive enable = SC0 O5's)
40–47	SC2 SA tap 0–7 (drive enable = SC2 O5's)

6.4 System block (0x1C00)

Offset	Register
0x00	ID (R)
0x01	SYS_CTRL: [0] main core run, [1] main core halt, [2] restart the prescaler
0x02	RUN_MASK: [3:0] start these sub-cores on the same clock, [7:4] halt these
0x03	IRQ: host IRQ status; host writes 1 to clear
0x04	DOORBELL: host sets, main core clears (WAIT bit 12)
0x05	EVENTS: the live WAIT vector (R)
0x10–0x15	CRC unit (§5.2)

6.5 Bring-up

1. rst_n clears every program and config latch. All sub-cores and the main core are halted; every pad is 'none'. A cleared program row is "state 0, wait forever, no actions", so nothing moves.
2. The host writes the SRAM image, then each sub-core's program and config (pairs: write the split image to both banks).
3. CTRL.SOFT_RESET on each sub-core.
4. The host writes the pin map.
5. The host releases the main core, or skips it and drives the FIFOs itself.
6. One RUN_MASK write starts the chosen sub-cores on the same clock.

Deselecting the design loses all state, so every reselect starts again at step 1.

6.6 Data flow

- **TX:** host → SRAM ring → main core (frames, CRC, stuffing, line code) → FIFO A → SA → pin (or SA taps → pads).
- **RX:** pin → SB → FIFO B → main core (checks, destuffing, decoding) → SRAM → host IRQ.
- **Capture:** event → CAP → FIFO B, same path as RX words.
- **In-band status:** tagged words. FIFO A tagged words are commands (START, command byte, raw sequence) seen by TXTAG or DISP; FIFO B tagged words (MARK) carry the state number of an error or end of frame, so it stays attached to the data it belongs to.

6.7 Assembler reference

State line: `LABEL : COND ? T : F | actions` or `LABEL : -> T | actions`. Actions: at most one pin action, at most two micro-ops, and `[PE]`. Pin actions: `Ok=0|1|Z|SA|AUX|IDLE`, `Ok^`, `LANES=code`, `PAT1`–`PAT5`, `ALL=Z`, `ALL=IDLE`. A condition is a name from §4.3 or a pin name (`ui4`, `uio1`).

Directive	Parameters
`.out Ok`	`pin=` pad, `mode=pp|od`, `idle=`, `inv=`, `init=IDLE|SA|AUX|Z`, `aux=`, `lane=`, `oe=lane`
`.evu Ek`	`pin=`, `edge=level|rise|fall|both`, `sticky=`, `qual=pin:level`
`.tmr T1|T2`	`per=`, `cmp=`, `rld=`, `psc=0..3`, `frac=` (T1), `rst=E0..E3|OTHERP|OTHERC`, `oneshot=`, `gate=`
`.cnt`	`pre=`, `dec=event`, `inc=event`, `wrap=`
`.shf SA|SB`	`order=lsb|msb`, `n=`, `lanes=1|2|4|8`, `ev=`, `stream=`; SA: `skip1=`, `udf=0|1|hold`; SB: `pin=` (first lane)
`.match`	`val=`, `mask=`
`.pre P0|P1`	`ev=`, `vec=label`, `always=`
`.ext`	`mask=src,src,…`, `mb=0..3`, `sig=0..7`, `sticky=`
`.disp`	`src=SB|FBNEW|FAHEAD|IN`, `base=`, `width=2|3`
`.pat Pk`	`Ox=code` for any subset of O0–O5 (`0 1 Z ^ SA AUX IDLE`)
`.tap`	`pins=firstpad:count` (SC0 and SC2)
`.sbit SB`, `.entry label`, `.split 24|32`, `.base 8`	

`pin=` on `.out` and `.tap` produce pin-layer settings; everything else is sub-core config.

---

7. Example programs

Every program here runs in the cycle model (`pe_arch1_model.zip`). Clock 50 MHz unless stated.

7.1 UART TX, framed by the main core: 2 states, any format

```
.out O0 pin=uo2 mode=pp idle=1 init=SA          ; TX follows SA permanently
.tmr T1 per=433 rld=1                           ; 434 clk/bit = 115200 Bd
.shf SA order=lsb n=10 ev=T1P stream=1 udf=1    ; frame = stop|data|start (n=11 for 8E1)
GO  : -> RUN | T1.GO, SA.LOAD
RUN : -> RUN                                    ; hardware streams frames back to back
```

The main core computes parity, builds the frame and feeds the CRC unit:

```
send:   SHI  r6, r4, SRL 4        ; even parity of r4
        XOR  r6, r6, r4
        SHI  r5, r6, SRL 2
        XOR  r6, r6, r5
        SHI  r5, r6, SRL 1
        XOR  r6, r6, r5
        ANDI r6, r6, 1
        SHI  r6, r6, SLL 9        ; parity -> bit 9
        SHI  r5, r4, SLL 1        ; data -> bits 8:1, start bit 0
        OR   r6, r6, r5
        LI   r5, 1
        SHI  r5, r5, SLL 10       ; stop bit -> bit 10
        OR   r6, r6, r5
        LI   r5, 1
        WAIT r5, r5               ; sleep until SC0 FIFO A has space
        ST   r6, [r1+0x04]        ; push frame (r1 = 0x1000)
        BLTU r2, r1, feed         ; buffer byte? feed the CRC
        JR   r7
feed:   LI   r5, 0
        LIH  r5, 0x1C
        ST   r4, [r5+0x13]        ; CRC_DATA (8 clocks, bit-serial)
        JR   r7
```

Result: "123456789" as 8E1, then the CRC-8/SMBUS byte 0xF4. Every bit is 434 clocks; frames are back to back.

7.2 UART RX 8N1: 8 states

```
.evu E0 pin=ui3 edge=fall
.tmr T2 per=216 rst=E0 oneshot=1        ; half bit, restarted by every falling edge
.tmr T1 per=433 rld=4                   ; bit clock, sampling at bit centre
.shf SB order=lsb n=8 ev=T1P pin=ui3
IDLE : E0   ? CHK  : IDLE
CHK  : T2P  ? VAL  : CHK
VAL  : ui3  ? IDLE : DATA               ; high at mid start bit = glitch
DATA : SBD  ? STOP : DATA  | T1.GO, SB.ARM
STOP : T1P  ? SCHK : STOP
SCHK : ui3  ? OK   : FERR
OK   : -> IDLE             | SB.PUSH
FERR : -> IDLE             | SB.PUSH, MARK   ; byte, then a tagged framing-error word
```

Samples land exactly on bit centres; ±2.5 % baud error decodes cleanly.

7.3 SPI controller, mode 0: 5 states

```
.out O0 pin=uo3 mode=pp idle=0 init=AUX aux=CLK1   ; SCK = T1 wave
.out O1 pin=uo4 mode=pp idle=0 init=SA             ; MOSI
.out O2 pin=uo5 mode=pp idle=1                     ; CS_n
.tmr T1 per=9 cmp=4 gate=1                         ; clk/10; burst ends when CNT = 0
.tmr T2 per=4 oneshot=1                            ; CS_n hold
.cnt pre=8 dec=T1P
.shf SA order=msb n=8 ev=T1P
.shf SB order=msb n=8 ev=T1C pin=ui4 stream=1
IDLE : TXNE ? LOAD : IDLE   | O2=1, SB.ARM
LOAD : -> XFER              | SA.LOAD, CNT.LDP
XFER : CNT0 ? NEXT : XFER   | O2=0, T1.GO
NEXT : TXNE ? LOAD : HOLD                           ; back-to-back bytes keep CS_n low
HOLD : T2P  ? IDLE : HOLD   | T2.GO
```

clk/4 (12.5 MHz) needs config changes only: `per=3 cmp=1` on T1, SB sampling on T2P with T2 restarted by T1C (`per=2 rst=OTHERC`), which absorbs the input round trip. At clk/4 MOSI has one clock (20 ns) of setup before SCK rises.

7.4 SPI target, mode 0: 3 states

```
.out O0 pin=uio2 mode=pp idle=1 init=Z             ; MISO, released while not selected
.evu E0 pin=ui5 edge=fall                          ; CS_n fall
.evu E1 pin=ui5 edge=rise                          ; CS_n rise
.evu E2 pin=ui6 edge=rise                          ; SCK rise: sample MOSI
.evu E3 pin=ui6 edge=fall                          ; SCK fall: next MISO bit
.shf SA order=msb n=8 ev=E3 stream=1 udf=1
.shf SB order=msb n=8 ev=E2 pin=ui7 stream=1
.pre P0 ev=E1 vec=ABORT always=1                   ; CS_n rising aborts from any state
.entry IDLE
ABORT: -> IDLE              | O0=Z, SA.STOP
IDLE : E0 ? XFER : IDLE     | SB.ARM
XFER : -> XFER              | O0=SA, SA.LOAD
```

7.5 I2C controller: 15 states

Handles clock stretching; speed is set by timer values alone.

Edge-ordering rule. The program drives only SCL. SDA always follows SA, and SA shifts on the *real* SCL fall seen at the pin, so a data bit changes on SDA only after SCL is low at the pad. The controller never changes SCL and SDA in the same clock. START and STOP are a single `SA.STEP` while SCL is held high.

FIFO A words are 12 bits:

Word	Layout
START + byte (tagged)	[11] 1 · [10] 0 (the START edge) · [9:2] byte · [1] ACK bit to send · [0] STOP_n
Byte only	[11:4] byte · [3] ACK bit to send · [2] STOP_n · [1:0] 11

The ACK bit to send is 1 to release SDA (writes) and 0 to ACK a read; 1 on the last read byte gives the NACK. STOP_n = 0 puts a STOP after this byte. Send 0xFF to read. FIFO B words are 9 bits: the byte seen on SDA, then the ACK bit seen.

```
.out O0 pin=uio0 mode=od idle=1                    ; SCL: the only pin the program drives
.out O1 pin=uio1 mode=od idle=1 init=SA            ; SDA: always the shifter's current bit
.evu E0 pin=uio0 edge=rise                         ; real SCL rise
.evu E1 pin=uio0 edge=fall                         ; real SCL fall
.tmr T1 per=249                                    ; tLOW, tSU;STA, tHD;STA (5 us)
.tmr T2 per=243 rst=E0                             ; tHIGH, tSU;STO, from the real rise
.cnt pre=9
.shf SA order=msb n=12 ev=E1 udf=1
.shf SB order=msb n=9 ev=E0 pin=uio1
IDLE : TXNE  ? CHK  : IDLE
CHK  : TXTAG ? LDS  : LDN  | SB.ARM                 ; tagged word = START first
LDS  : -> STB              | SA.LOAD, CNT.LDP       ; SA shows 1: SDA released
LDN  : -> LOW              | SA.LOAD, CNT.LDP       ; SA shows b7 while SCL is low
STB  : uio0  ? STC  : STB  | O0=1                    ; release SCL, wait until high
STC  : T1P   ? STD  : STC  | T1.GO                   ; tSU;STA
STD  : T1P   ? STE  : STD  | SA.STEP, T1.GO          ; SDA falls with SCL high = START
STE  : -> LOW              | O0=0, SB.ARM
LOW  : T1P   ? HIGH : LOW  | T1.GO                   ; tLOW
HIGH : T2P   ? FALL : HIGH | O0=1, T2.STOP, CNT.DEC ; tHIGH from the real rise
FALL : CNT0  ? ENDB : LOW  | O0=0
ENDB : T1P   ? EB3  : ENDB | SB.PUSH, T1.GO          ; SA now shows STOP_n
EB3  : SBIT  ? IDLE : STP
STP  : T2P   ? STR  : STP  | O0=1, T2.STOP           ; tSU;STO
STR  : -> IDLE             | SA.STEP                ; SDA rises with SCL high = STOP
```

Model results (write, then repeated-START read with ACK and NACK, target stretching 14 µs): no clock changes both lines; SDA moves while SCL is high only for the 3 STARTs and 2 STOPs; at 100 kHz tLOW ≥ 253 and tHIGH ≥ 250 clocks. With T1 = 69 and T2 = 47 it runs at 382–394 kHz meeting tLOW and tHIGH.

7.6 I2C multi-controller with arbitration: 24 states (split 24/8)

SC0 borrows 8 rows from SC1, which keeps rows 8–15 and runs UART TX at the same time. FIFO A words carry a 2-bit type in [13:12] that one DISP state decodes: 0 write byte (arbitrated), 1 START + byte, 2 read byte, 3 release the bus.

```
.split 24
.out O0 pin=uio0 mode=od idle=1                    ; SCL
.out O1 pin=uio1 mode=od idle=1 init=SA            ; SDA
.evu E0 pin=uio0 edge=rise
.evu E1 pin=uio0 edge=fall
.evu E2 pin=uio1 edge=rise qual=uio0:1             ; a STOP on the bus
.tmr T1 per=249
.tmr T2 per=243 rst=E0
.cnt pre=9
.shf SA order=msb n=12 ev=E1 udf=1
.shf SB order=msb n=9 ev=E0 pin=uio1
.disp src=FAHEAD base=12 width=2                   ; word type in bits 13:12
IDLE : TXNE ? CHK  : IDLE
CHK  : -> LDW              | DISP, SB.ARM           ; 4-way on the word type
STB  : uio0 ? STC  : STB   | O0=1
STC  : T1P  ? STD  : STC   | T1.GO
LDW  : -> WLOW             | SA.LOAD, CNT.LDP       ; type 0
LDS  : -> STB              | SA.LOAD, CNT.LDP       ; type 1
LDR  : -> RLOW             | SA.LOAD, CNT.LDP       ; type 2
LOST : -> BUSY             | ALL=IDLE, MARK         ; type 3, or arbitration lost
STD  : T1P  ? STE  : STD   | SA.STEP, T1.GO
STE  : -> WLOW             | O0=0, SB.ARM
WLOW : T1P  ? WHI  : WLOW  | T1.GO
WHI  : T2P  ? AR1  : WHI   | O0=1, T2.STOP, CNT.DEC
AR1  : CNT0 ? WFAL : AR2                           ; ACK slot: not arbitrated
AR2  : SBIT ? AR3  : WFAL                          ; we drive 0: nothing to check
AR3  : uio1 ? WFAL : LOST                          ; we released SDA and it is low: lost
WFAL : CNT0 ? ENDB : WLOW  | O0=0
RLOW : T1P  ? RHI  : RLOW  | T1.GO
RHI  : T2P  ? RFAL : RHI   | O0=1, T2.STOP, CNT.DEC
RFAL : CNT0 ? ENDB : RLOW  | O0=0
ENDB : T1P  ? EB3  : ENDB  | SB.PUSH, T1.GO
EB3  : SBIT ? IDLE : STP
STP  : T2P  ? STR  : STP   | O0=1, T2.STOP
STR  : -> IDLE             | SA.STEP
BUSY : E2   ? IDLE : BUSY  | SA.STOP                ; wait for the winner's STOP
```

Partner program on SC1 (`.base 8`): the 2-state UART TX of §7.1. In the model a competing controller wins at bit 1; SC0 lets go of both lines, MARKs, waits for the STOP; the main core flushes FIFO A and re-queues; the retry completes (bus S, P, S, AW, P). The UART on SC1 sends its 3 bytes meanwhile.

7.7 JTAG: 5 states

FIFO A: a TCK count, then words of 8 (TMS, TDI) pairs. FIFO B: 8-bit TDO words.

```
.out O0 pin=uo2 mode=pp idle=1 init=SA            ; TMS = lane 0
.out O1 pin=uo3 mode=pp idle=0 init=SA            ; TDI = lane 1
.out O2 pin=uo4 mode=pp idle=0 init=AUX aux=CLK1  ; TCK
.tmr T1 per=9 cmp=4 gate=1
.cnt dec=T1P
.shf SA order=lsb n=16 lanes=2 ev=T1P stream=1 udf=hold
.shf SB order=lsb n=8 ev=T1C pin=ui5 stream=1     ; TDO at TCK rise
IDLE : TXNE ? CNTW : IDLE
CNTW : -> GO               | CNT.LDF, SB.ARM
GO   : -> RUN              | SA.LOAD, T1.GO
RUN  : CNT0 ? FIN  : RUN
FIN  : -> IDLE             | SB.PUSH              ; partial TDO word
```

TAP navigation is data, so the program never changes. The model reads IDCODE 0x4BA00477 and writes 0xDEADBEEF to DR.

7.8 USB low-speed transmit: 4 states

D+/D− are two SA lanes; J, K and SE0 are data. The main core does NRZI, bit stuffing and EOP.

```
.out O0 pin=uio0 mode=pp idle=0 init=Z            ; D+
.out O1 pin=uio1 mode=pp idle=1 init=Z            ; D-
.tmr T1 per=31 rld=1                              ; 32 clk/bit at 48 MHz
.shf SA order=lsb n=16 lanes=2 ev=T1P stream=1 udf=hold
IDLE : TXNE ? GO   : IDLE  | ALL=Z
GO   : -> DRV              | SA.LOAD, T1.GO
DRV  : SAD  ? REL  : DRV   | LANES=SA             ; both lines from the same clock
REL  : T1P  ? IDLE : REL
```

Every symbol is exactly 32 clocks, no SE1 appears, and the bus is released after the final J.

7.9 SWD host: 16 states

SWDIO is a (value, drive-enable) lane pair, so turnarounds are data. FIFO A: a cycle count (tagged = raw sequence such as line reset, untagged = packet), then words of 8 (value, drive) pairs. FIFO B: 12-bit words of SWDIO samples; word 0 of a packet is request echo, turnaround and ACK.

```
.out O0 pin=uio2 mode=pp idle=0 init=AUX aux=CLK1        ; SWCLK
.out O1 pin=uio3 mode=pp idle=0 init=SA lane=0 oe=lane    ; SWDIO: value lane 0, drive lane 1
.tmr T1 per=9 cmp=4 gate=1
.cnt dec=T1P
.shf SA order=lsb n=16 lanes=2 ev=T1P stream=1 udf=hold
.shf SB order=lsb n=12 ev=T1C pin=uio3 stream=1
.disp src=FBNEW base=9 width=2                           ; {ACK1, ACK0}
IDLE : TXNE  ? CW   : IDLE
CW   : TXTAG ? RAW  : PKT   | SB.ARM
RAW  : -> RGO               | CNT.LDF
RGO  : -> RUN               | SA.LOAD, T1.GO
AFLT : -> ERR                                             ; 00 FAULT
AOK  : -> RUN                                             ; 01 OK: data phase streams on
AWT  : -> ERR                                             ; 10 WAIT
ANON : -> ERR                                             ; 11 no target
RUN  : CNT0  ? FIN  : RUN
FIN  : -> IDLE              | SB.PUSH
PKT  : -> PGO               | CNT.LDF
PGO  : -> HDR               | SA.LOAD, T1.GO
HDR  : SBD   ? ACK  : HDR
ACK  : -> AFLT              | DISP                       ; 4-way on the ACK bits
ERR  : -> STOP              | T1.STOP, MARK
STOP : -> IDLE              | HALT                       ; main core: soft reset, retry
```

Model: line reset, DPIDR read (0x2BA01477, parity good), a write, a WAIT answered by HALT → soft reset → retry, then reads. Zero contention cycles.

7.10 8080 parallel write: 8 states

```
.out O0 pin=uo2 mode=pp idle=1                    ; WR_n
.out O1 pin=uo3 mode=pp idle=1                    ; D/C_n
.out O2 pin=uo4 mode=pp idle=1                    ; CS_n
.out O5 mode=pp idle=0                            ; no pad: its drive enable is the bus direction
.tap pins=uio0:8                                  ; uio0..7 <- SA lanes 0..7
.tmr T1 per=4                                     ; 100 ns WR_n phases
.shf SA order=lsb n=8 lanes=8 ev=T1P udf=hold
.pat P1 O1=0 O2=0                                 ; command: D/C_n and CS_n low, same clock
.pat P2 O1=1 O2=0                                 ; data
.pat P3 O1=1 O2=1                                 ; deselect
IDLE : TXNE  ? CHK  : IDLE
CHK  : TXTAG ? CMD  : DAT                         ; tagged word = command byte
CMD  : -> WRL               | PAT1, SA.LOAD
DAT  : -> WRL               | PAT2, SA.LOAD
WRL  : T1P   ? WRH  : WRL   | O0=0, T1.GO
WRH  : T1P   ? NXT  : WRH   | O0=1, T1.GO           ; WR_n rise latches the byte
NXT  : TXNE  ? CHK  : END
END  : -> IDLE              | PAT3
```

7.11 NEC IR receive: 8 states

```
.evu E0 pin=ui4 edge=fall                          ; mark starts (receiver output low)
.evu E1 pin=ui4 edge=rise
.evu E2 pin=ui4 edge=both
.tmr T1 per=49                                     ; 1 us tick
.tmr T2 per=37499 psc=1 rst=E2 oneshot=1           ; 12 ms without an edge = end of frame
.cnt inc=T1P                                       ; counts microseconds, saturates
.ext mask=E0,T2P                                   ; next mark OR timeout
IDLE : E0   ? M0   : IDLE
M0   : -> WR               | CNT.LDP, T1.GO
WR   : E1   ? SP   : WR
SP   : -> WF               | CAP, CNT.LDP          ; mark width
WF   : EXT  ? CK   : WF
CK   : ui4  ? END  : MK                            ; line high: it was the timeout
MK   : -> WR               | CAP, CNT.LDP          ; space width
END  : -> IDLE             | MARK
```

The main core classifies widths. Model: leader 8999/4500 µs, 32 bits, address 0x04, command 0x45, end marker.

7.12 Quadrature decoder (×1): 3 states

```
.evu E0 pin=ui4 edge=rise qual=ui5:0               ; A rises, B low: forward
.evu E1 pin=ui4 edge=rise qual=ui5:1               ; A rises, B high: backward
.cnt inc=E0 dec=E1 wrap=1
.ext mask=MB mb=0                                  ; snapshot request from the main core
RUN  : EXT  ? SNAP : RUN
SNAP : -> HOLD             | CAP
HOLD : EXT  ? HOLD : RUN
```

The position counts in hardware with no program involvement. Model: +37, −12, −30 steps read back as 37, 25, −5.

---

8. Protocol coverage and slot usage

Verified in the cycle model (12 programs, 101 states)

Program	States	New 1.0 features used
UART TX (main-core framing, any format)	2	—
UART RX 8N1 with framing-error marker	8	—
SPI controller mode 0 (clk/10, clk/4)	5	—
SPI target mode 0	3	—
I2C controller with stretching, 100/400 kHz	15	—
I2C multi-controller with arbitration	24	elastic split 24/8, DISP, ALL=IDLE
JTAG	5	2 lanes, underrun hold
USB-LS TX	4	2 lanes, underrun hold, ALL=Z
SWD host with WAIT/FAULT handling	16	DISP, OE from lane, 2 lanes, underrun hold
8080 parallel write	8	patterns, 8 lanes, SA taps
NEC IR receive	8	CAP, counter up, EXT mask
Quadrature decoder	3	CAP, counter up/down, EXT mask

Slot usage over those 101 states

Pin actions	Micro-ops	States
0	0	33
0	1	21
0	2	23
1	0	8
1	1	13
1	2	3

No state needs a second pin slot or a third micro-op.

Expected to fit, not yet modelled (the mapping is argued, not simulated)

Protocol	How it maps
I2C target	Matcher on the address byte, SDA by SA with OE from the open-drain mode; v0.3 program to be ported
1-Wire controller	Open-drain output, T1 slot timing, SB samples at the read point; v0.3 program to be ported
WS2812 / DShot	Per bit: drive high, branch on SBIT to wait for a short or long high time (T1C or T2P), drive low until T1P; v0.3 program to be ported
MDIO (clause 22)	As SWD: (value, drive) lane pair for the turnaround
3-wire SPI, Microwire	As SPI controller, with OE from lane on the shared data line
Quad-SPI controller	8 lanes as four (value, drive) pairs on O0–O3, SCK on O4, CS_n on O5
6800 parallel	As 8080 with an E strobe instead of WR_n
LIN, DMX512, ISO 7816	UART TX/RX plus a break or guard time from T2
PS/2 device and host	Device-clocked: shift on clock-pin events, open-drain lines
PWM, servo, stepper STEP/DIR	Timers with wave outputs; a pattern sets direction and enable together
Manchester / 10BASE-T TX	AUX = SA ⊕ CLK1 at 40 MHz (pads uncharacterised)
Pulse-width / capture protocols (SENT RX, ultrasonic echo)	CAP and EXT mask, as NEC

Hard or out of reach: USB-LS device (the handshake turnaround with a software CRC is tight); CAN with a real-time CRC-dependent ACK (needs an optimistic ACK; destuffing costs states); 10BASE-T RX (5 clocks per bit); quadrature ×2/×4 (one inc and one dec event select only).

9. Verification status

The model package runs these benches; all pass.

Bench	What it checks
test_uart.py, test_uart_detail.py	TX 8N1/8E1, back to back, exact 434-clock bits; RX at −2.5/0/+2.5 % baud, glitch reject, framing-error MARK, sample point on bit centre
test_spi.py, test_spi_fast.py	Controller at clk/10 and clk/4, full duplex, CS_n held across bytes
test_spi_slave.py	Target with CS_n abort by preemption; MISO released after CS_n
test_i2c.py	Controller write and repeated-START read, stretching, edge audit, 100 and 400 kHz timing
test_i2c_multi.py	Elastic 24/8 split, arbitration loss, BUSY wait, flush and retry, UART on the partner
test_jtag_usb.py	JTAG TAP walk, IDCODE, DR write; USB-LS symbols, no SE1
test_swd.py	SWD line reset, read, write, WAIT → HALT → retry, no contention

Bench	What it checks
test_parallel_capture.py	8080 write with patterns and taps; NEC IR capture; quadrature up/down
test_pins.py	6 pads on one edge from lanes + toggle + NPART; one pattern changes 6 outputs on one edge; 8 taps + 6 outputs = 14 pads on one edge
test_system.py	Main core + sub-core: 8E1 framing, parity and CRC-8 trailer in software; CRC-8, CRC-16, CRC-32 check values

---

10. Open items and risks

- **Area estimate ±15 %.** The first hardening milestone is one sub-core pair through the TT flow: area, latch-array STA (including the partner read path) and Metal2–4 congestion. If it comes in high, the levers in order are: FIFO A 4 → 2 (−1.0 point), patterns 5 → 3 (≈−0.5 point), then 8×4 tiles (37.2 %).
- **Encoding space is full.** All 32 conditions and all 32 micro-ops are assigned. A future addition needs a config-selected alternate op bank or a wider OP field.
- **Pads.** CMOS5L pads are uncharacterised, so SPI faster than clk/10, 100 ns 8080 phases and 10BASE-T TX depend on silicon.
- **uio read-back while driving.** The I2C programs see their own SCL fall through the pad. If read-back proves unreliable, LOOPBACK provides the internal value (it cannot see another device stretching).
- **Main-core throughput.** A tight SRAM-to-FIFO copy loop moves about one word per 7 clocks. Streams faster than that (8 lanes at a few clocks per shift) run in bursts of 4 words; everything in §7 is well below it.
- **Not yet ported to 1.0:** I2C target, 1-Wire, WS2812 from v0.3.
- **USB-LS device** needs the 32-state split plus the main core; the handshake turnaround with a software CRC is the tightest point.
- **Path to Idea 2**, if ever needed: only the sequencer and the control half of the instruction change. The action half, the datapath, the bus window and everything outside the sub-core stay the same.